message-relay — Business Requirements Document

> **Version:** 1.0 | **Status:** Draft | **Author:** Prashant Verma | **Domain:** Platform Engineering · Messaging & Delivery

---

Table of Contents

---

1. Executive Summary

Customer-facing notifications — OTP codes, payment confirmations, alerts — are among the most time-sensitive and trust-critical communications a platform sends. When these fail silently, customers lose trust and support tickets spike. When they fail loudly but without recovery, engineers lose sleep. When they cannot be audited, compliance and disputes become unresolvable.

message-relay is the delivery engine that consumes notification events published by rate-sentinel, validates their integrity, routes them to the correct vendor channel, tracks every delivery attempt in a durable audit store, and ensures that no message is ever silently lost — through automatic retries, circuit breaking, dead-letter queuing, and real-time alerting.

This document defines the business requirements that govern what message-relay must do, why, and how success is measured — independent of technical implementation details.

---

2. Problem Statement

2.1 Silent Delivery Failures

Without a dedicated delivery engine, notification failures are swallowed by individual service implementations — there is no visibility into what failed, why, or how many customers were affected. Operations teams discover failures only when customers complain.

2.2 No Recovery Mechanism for Transient Vendor Failures

Vendor APIs (SMS gateways, email providers, WhatsApp Business API) experience intermittent outages. Without retry logic and circuit breaking, a 30-second vendor blip silently drops thousands of notifications with no recovery path.

2.3 No Audit Trail for Regulatory and Support Use

When a customer disputes "I never received the OTP" or "I wasn't notified about the failed payment," there is no single source of truth to confirm what was sent, when, to which channel, and whether it succeeded. This creates unresolvable disputes and compliance exposure.

2.4 Tight Coupling Between Business Services and Vendors

Each business service maintaining its own vendor SDK integrations means vendor changes (API version updates, credential rotation, switching providers) require coordinated changes across multiple services. There is no way to swap a vendor without touching business code.

2.5 PII Exposure in Operational Systems

Phone numbers and email addresses passed through logging, monitoring, and operations tooling create PII exposure risk. Without a systematic masking layer, engineers working in production systems can inadvertently access or leak customer contact data.

---

3. Stakeholders

| Stakeholder | Role | Primary Concern |

|---|---|---|

| Platform / DevOps Engineer | Operates delivery infrastructure | Consumer lag visibility, DLQ monitoring, zero data loss |

| Business / Product Owner | Owns delivery SLA and vendor cost | 99.9%+ delivery rate, bounded vendor spend, no silent failures |

| Customer Support Agent | Resolves customer disputes | Searchable audit trail: was the message sent? did it succeed? |

| Security / Compliance Officer | Data protection and governance | PII masking, DLQ auditability, no raw contact data in logs |

| End Customer | Receives the notification | Fast, reliable delivery; no duplicate messages |

| rate-sentinel (upstream system) | Publishes notification events | Stable Kafka contract; events are processed exactly once |

---

4. Business Objectives

BO-1 · Guarantee at-least-once notification delivery

Every notification event published to Kafka must be processed and delivery attempted at least once. No message may be silently dropped due to application restart, instance crash, or transient infrastructure failure.

Acceptance Criteria:

• Manual Kafka acknowledgement ensures no message is lost on application crash

• Consumer lag stays below 50 messages in steady-state operation

• After any planned or unplanned restart, all unacknowledged messages are redelivered and processed

---

BO-2 · Self-heal from transient vendor failures without manual intervention

Short-lived vendor outages (< 30 seconds) must be absorbed automatically. Operations teams must not need to intervene for transient failures; they must only be alerted for persistent failures that exceed the retry budget.

Acceptance Criteria:

• Resilience4j circuit breaker opens on vendor failure, pauses delivery, and closes automatically on vendor recovery

• Messages queued during an outage are delivered in order after recovery with no data loss

• Engineers are only alerted (via Sentry) when a message exhausts all retry attempts and enters DLQ — not for individual transient failures

---

BO-3 · Provide a durable, searchable audit trail for every notification

Operations, support, and compliance teams must be able to answer: was this message sent? when? via which channel? how many attempts were made? did it succeed or fail? — for any message, at any time, from a single API.

Acceptance Criteria:

• Every delivery attempt is persisted to MongoDB with a complete `statusHistory` array

• Elasticsearch enables sub-100ms search by recipient, status, channel, clientId, and date range

• `GET /api/messages/{eventId}` returns the full lifecycle of any notification

• `GET /api/messages/stats/dlq` provides a real-time delivery health summary

BO-4 · Enable vendor-agnostic channel delivery

The business must be able to switch SMS providers, email vendors, or add new channels without touching consumer or delivery orchestration logic. Vendor selection is an operational concern, not a code change.

Acceptance Criteria:

• Replacing SmsSender implementation (e.g. Twilio → Gupshup) requires zero changes to SmsConsumer, DeliveryService, or any other class

• Adding a new channel requires only a new MessageSender implementation — no modifications to existing code

• All senders are independently testable with mock implementations; no real vendor calls required in CI

BO-5 · Protect customer PII in all operational contexts

Customer contact data (phone numbers, email addresses) must never appear unmasked in logs, monitoring dashboards, Sentry events, or search results accessible to engineers and support staff.

Acceptance Criteria:

• All stored `recipient` fields show masked values only (`+91****7890`, `pr**@domain.com`)

• No unmasked phone number or email address appears in any log line

• Sentry DLQ alerts contain masked recipient only

• Search API returns masked recipient values — raw values never exposed through any API endpoint

---

5. Business Rules

BR-1 · Invalid payloads must be dead-lettered immediately without retry

A structurally invalid message — missing required fields, malformed recipient, unknown channel — will never become valid through retrying. Retrying it wastes vendor API calls, inflates DLQ noise, and delays processing of valid messages behind it in the queue.

• `PayloadValidator` failure immediately routes to `DlqHandler` — no retry attempt made

• DLQ publish includes the validation error details as the failure reason

• `ack.acknowledge()` is still called to prevent re-consumption of a permanently invalid message

---

BR-2 · Delivery records must be append-only

The `statusHistory` array is the authoritative audit record of what happened to a notification. It must reflect the true sequence of events without any retroactive modification.

• Status transitions are appended to `statusHistory`, never overwritten

• A record that reaches `DEAD_LETTERED` is never updated again — it is a terminal state

• Historical status entries must include the timestamp and reason for every transition

---

BR-3 · Elasticsearch sync failures must not block delivery

Elasticsearch is a secondary index for search convenience, not the system of record. An Elasticsearch outage must not prevent message delivery or cause consumer lag accumulation.

• `indexToElasticsearch()` failures are caught, logged with a warning, and skipped

• Delivery continues and completes regardless of Elasticsearch availability

• MongoDB remains the authoritative source; Elasticsearch is rebuilt from MongoDB on recovery

---

BR-4 · DLQ events must trigger immediate Sentry alert

A dead-lettered message represents a notification the customer did not receive. Operations teams must know about this within seconds — not minutes — to assess impact and initiate manual replay if necessary.

• Every `DlqHandler.sendToDlq()` call must publish a Sentry `ERROR`-level event synchronously

• Sentry event must include: `eventId`, `channel`, `clientId`, failure reason, and attempt count

• Sentry alert failure (network issue) must not prevent the DLQ Kafka publish from succeeding

---

BR-5 · At-least-once delivery requires idempotent processing

Manual Kafka acknowledgement and retry on crash means the same message may be consumed more than once. The delivery flow must tolerate re-processing without creating duplicate vendor calls or duplicate MongoDB records.

• On startup, consumers check `DeliveryRecordRepository.findByEventId()` before processing

• If a record already exists in a terminal state (`SENT`, `DEAD_LETTERED`), processing is skipped

• `ack.acknowledge()` is still called to prevent infinite re-consumption

BR-6 · Consumer concurrency must not exceed partition count

Running more consumer threads than topic partitions wastes threads and can cause uneven load distribution. Concurrency configuration must align with partition count.

• Default concurrency: 3 threads per consumer (matching 3 partitions per topic)

• Increasing partitions requires corresponding increase in consumer concurrency configuration

• Concurrency settings are externally configurable without code changes

---

6. Functional Scope

| Feature Area | In Scope | Notes |

|---|---|---|

| Kafka consumption | 4 topics: SMS, Email, WhatsApp, Payment | Manual ack, 3 threads per consumer |

| Payload validation | Schema, format, PII masking | Invalid → DLQ immediately |

| Message delivery | SMS, Email, WhatsApp | Pluggable Factory pattern |

| Delivery state tracking | PENDING → SENDING → SENT / DEAD_LETTERED | Full statusHistory in MongoDB |

| Retry & circuit breaking | Resilience4j, 3 attempts, exponential backoff | Circuit opens at 50% failure rate |

| Dead letter queue | topic.dlq publish + Sentry alert | DlqConsumer monitors for replay |

| Audit storage | MongoDB (primary) | Full records, status history |

| Search index | Elasticsearch (secondary) | Fast keyword search |

| Search API | GET /api/messages/search | Paginated, multi-filter |

| Stats API | GET /api/messages/stats/dlq | Real-time delivery health |

| Observability | Prometheus metrics, structured logs | Grafana dashboard |

---

7. Performance Requirements

| Requirement | Target | Measurement Method |

|---|---|---|

| Message delivery throughput | > 3,500 msg/sec (mixed channels) | JMeter Kafka producer + consumer lag monitoring |

| Payload validation throughput | > 25,000 msg/sec | JMeter direct validator benchmark |

| Elasticsearch search latency | p95 < 100ms | JMeter search API load test |

| MongoDB write latency | p99 < 12ms | Micrometer timer on repository writes |

| Consumer lag at steady state | < 50 messages | Kafka consumer group lag metric in Grafana |

| DLQ alert latency | < 5 seconds from failure | Measured from exhausted retry to Sentry event received |

---

8. Security & Compliance Requirements

SC-1 · PII masking at point of ingestion

Phone numbers and email addresses must be masked before any write to MongoDB, Elasticsearch, or any log output. Masking must occur in `PayloadValidator` before `DeliveryService` creates the `DeliveryRecord`.

SC-2 · Raw recipient stored with restricted access

The unmasked recipient is stored in `recipientRaw` on the `DeliveryRecord` for operational use (e.g. re-sending). This field must not be exposed through any search, stats, or list API endpoint. Access requires direct database query with appropriate authorisation.

SC-3 · No vendor credentials in source code

Vendor API keys (Twilio Auth Token, SendGrid API Key, WhatsApp token) must be injected via environment variables or Azure Key Vault in production. Checkmarx scan must flag any hardcoded credential as a blocking finding.

SC-4 · SAST scan on every pull request

Checkmarx One free-tier scan configured as a required GitHub Actions step. Zero tolerance for critical or high-severity findings on merge.

SC-5 · DLQ payload must not contain unmasked PII

Messages published to `topic.dlq` contain the original `NotificationEvent`. The `recipient` field in this event comes from rate-sentinel and may be unmasked. message-relay must not re-publish raw recipient values in Sentry alerts — only the masked version derived by `PayloadValidator` may appear in alerts.

---

9. Operational Requirements

OR-1 · Delivery failures visible without database queries

The Grafana dashboard must show current delivery success rate, DLQ depth, consumer lag per topic, and p99 delivery latency — all derived from Prometheus metrics. Operations staff must never need to run a MongoDB query to understand system health.

OR-2 · DLQ messages available for manual replay

Dead-lettered messages must remain in `topic.dlq` and MongoDB with their original payload intact, available for manual replay via a future `POST /api/dlq/{eventId}/replay` endpoint. No messages are automatically purged from the DLQ within the 7-day Kafka retention window.

OR-3 · Consumer group isolation for monitoring

The `DlqConsumer` must use a separate consumer group (`dlq-monitor-group`) from the main `delivery-engine-group`. Monitoring activity must never affect the consumer lag or throughput of the primary delivery consumers.

OR-4 · Single-command local environment

A new engineer must be able to run the complete delivery engine locally — including Kafka, MongoDB, Elasticsearch, and Grafana — using `docker-compose up -d` followed by `./mvnw spring-boot:run` with no additional setup.

OR-5 · Graceful shutdown with in-flight message completion

On `SIGTERM` (Kubernetes pod shutdown), the application must complete processing of all in-flight messages before stopping consumers. No message acknowledgement may be lost during rolling deployment.

---

10. Success Metrics & KPIs

| KPI | Target | Baseline (pre-relay) |

|---|---|---|

| Delivery success rate (30-day rolling) | ≥ 99.9% | Untracked (silent failures) |

| Silent delivery failures | 0 | Unknown (no audit trail) |

| Time to detect a delivery failure | < 5 seconds | Minutes to hours (log trawling) |

| Support ticket resolution time (delivery disputes) | < 2 minutes | 30+ minutes (no audit trail) |

| Vendor switch lead time | < 1 sprint | 2–3 sprints (service-by-service) |

| Consumer lag (steady state) | < 50 messages | N/A |

| DLQ depth (daily average) | < 10 messages | N/A |

---

11. Assumptions & Constraints

Assumptions:

• rate-sentinel publishes well-formed `NotificationEvent` objects; message-relay validates but does not transform business logic

• Kafka topics (`topic.sms`, `topic.email`, `topic.whatsapp`, `topic.payment`, `topic.dlq`) are created by rate-sentinel on startup

• Vendor SDK integrations (Twilio, SendGrid, WhatsApp Business API) are mocked in v1.0; real integrations are a subsequent iteration

• MongoDB and Elasticsearch are available as separate infrastructure; message-relay does not manage their provisioning

• Eventual consistency between MongoDB and Elasticsearch (seconds delay) is acceptable for the search use case

Constraints:

• Java 17 and Spring Boot 3.x are the mandated runtime stack

• Elasticsearch version 8.x required for Spring Data Elasticsearch compatibility

• SonarCloud free tier requires a public repository for unlimited scans

• Checkmarx One free tier limits to 10 scans per month — scan must run on every PR to `main` only

• Sentry free tier: 5,000 errors/month; DLQ volume must stay within this budget or a paid tier is required

---

12. Out of Scope

The following are explicitly not part of message-relay v1.0:

• Delivery status webhook callbacks to confirm actual customer receipt (planned v1.1)

• DLQ replay REST endpoint (planned v1.1 — `POST /api/dlq/{eventId}/replay`)

• Scheduled batch retry of DLQ messages (manual replay only in v1.0)

• Real vendor SDK integrations (mocked in v1.0)

• Multi-region active-active consumer deployment

• In-app push notification delivery (Firebase, APNs)

• Message deduplication for at-most-once delivery (at-least-once is sufficient for v1.0)

• Elasticsearch index auto-rebuild from MongoDB (manual operation only)

---

13. Risk Register

| ID | Risk | Likelihood | Impact | Mitigation |

|---|---|---|---|---|

| R-01 | Kafka broker outage causes consumer lag accumulation and notification delay | Low | High | Kafka replication factor ≥ 2 in prod; consumer lag alert at 1,000 messages; consumers auto-resume on broker recovery |

| R-02 | MongoDB outage prevents delivery record creation — messages cannot be delivered | Medium | High | Retry on MongoDB write failure; circuit breaker on repository calls; alert if MongoDB unavailable > 30s |

| R-03 | Elasticsearch diverges from MongoDB after partial failure — stale search results | Medium | Medium | Elasticsearch sync failures logged and alerted; manual reindex procedure documented; "results may be delayed" noted in API docs |

| R-04 | Sentry quota exhaustion silences DLQ alerts — failures go unnoticed | Medium | High | Monitor Sentry usage in Grafana; alert at 80% monthly quota; upgrade to paid tier if DLQ rate is high |

| R-05 | Vendor SDK breaking change silences delivery without error — `SENT` status recorded but message not actually delivered | Medium | Medium | Vendor message ID captured; delivery receipt webhook (v1.1) will provide end-to-end confirmation |

| R-06 | Consumer rebalance during peak load causes temporary processing pause | Low | Low | Kafka `session.timeout.ms` tuned to 30s; consumer group stabilises automatically after rebalance |

---

14. Glossary

| Term | Definition |

|---|---|

| Consumer Group | A named group of Kafka consumers that collectively consume a topic, with each partition assigned to exactly one consumer in the group at any time |

| Manual Acknowledgement | A Kafka consumption mode where the consumer explicitly calls `ack.acknowledge()` after processing, preventing message loss on crash |

| Dead Letter Queue (DLQ) | A Kafka topic (`topic.dlq`) that receives messages that could not be delivered after exhausting all retry attempts |

| Circuit Breaker | A Resilience4j pattern that stops calling a vendor after a failure threshold is exceeded, allowing it to recover before resuming calls |

| Exponential Backoff | A retry strategy where wait time doubles between attempts (2s $\rightarrow$ 4s $\rightarrow$ 8s) to avoid overwhelming a recovering vendor |

| DeliveryRecord | The MongoDB document tracking the full lifecycle of a single notification event, including all status transitions |

| DeliveryDocument | The Elasticsearch document derived from DeliveryRecord, containing flattened fields optimised for keyword and full-text search |

| StatusHistory | An append-only array on DeliveryRecord recording every state transition with timestamp and reason |

| PII Masking | The process of replacing sensitive personal data (phone numbers, emails) with partially obscured equivalents before storage or logging |

| MessageSenderFactory | The Spring component that resolves the correct MessageSender implementation at runtime based on the notification channel |

| at-least-once delivery | A delivery guarantee ensuring every message is processed at least once; duplicates are possible on crash/retry and must be handled idempotently |